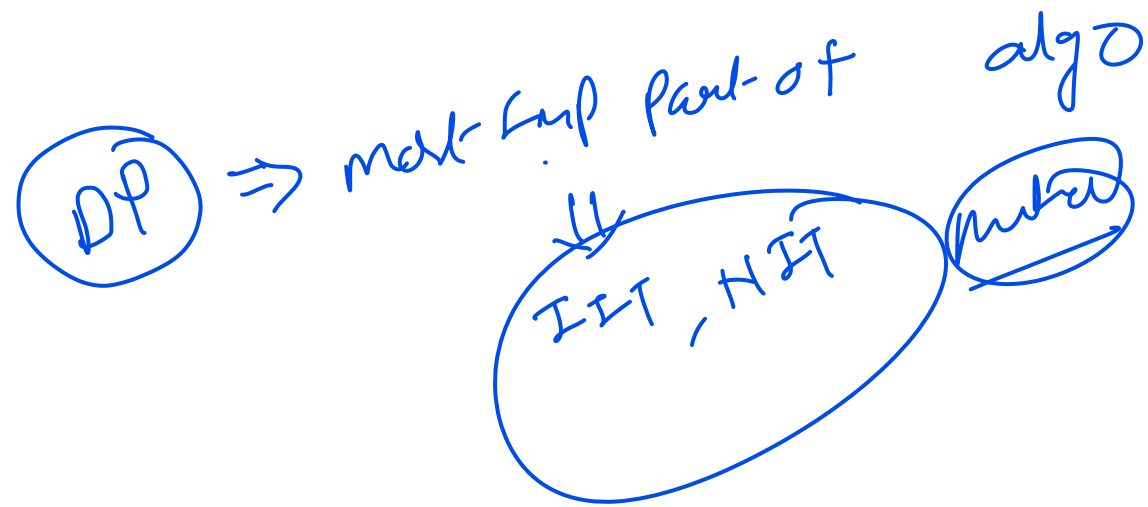




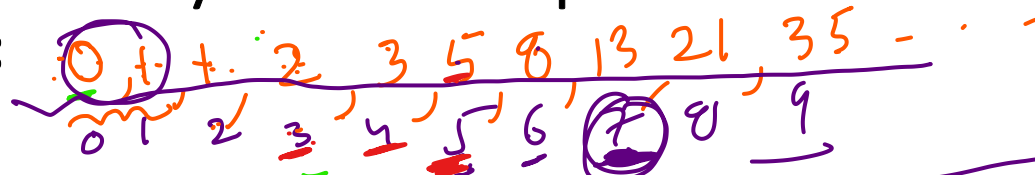
Lec_71

Introduction to DP (Dynamic Programming)

- try all combinations and do not repeat calculated things

Lets Understand by an Example

• Fibonacci Series:



* $\boxed{\begin{array}{l} \text{fib}(0) = 0 \\ \text{fib}(1) = 1 \end{array}}$

* $\boxed{\text{fib}(n) = \text{fib}(n-1) + \text{fib}(n-2)}$

$\text{fib}(7) = \text{fib}(6) + \text{fib}(5)$

```
fib(n)
{
  if (n == 0 || n == 1)
    return n;
  a = fib(n-1);
  b = fib(n-2);
  return a + b;
}
```

$\Rightarrow$ Base condition of the recursive

This program is without any sum

recursion: fn calling itself is called recursion.

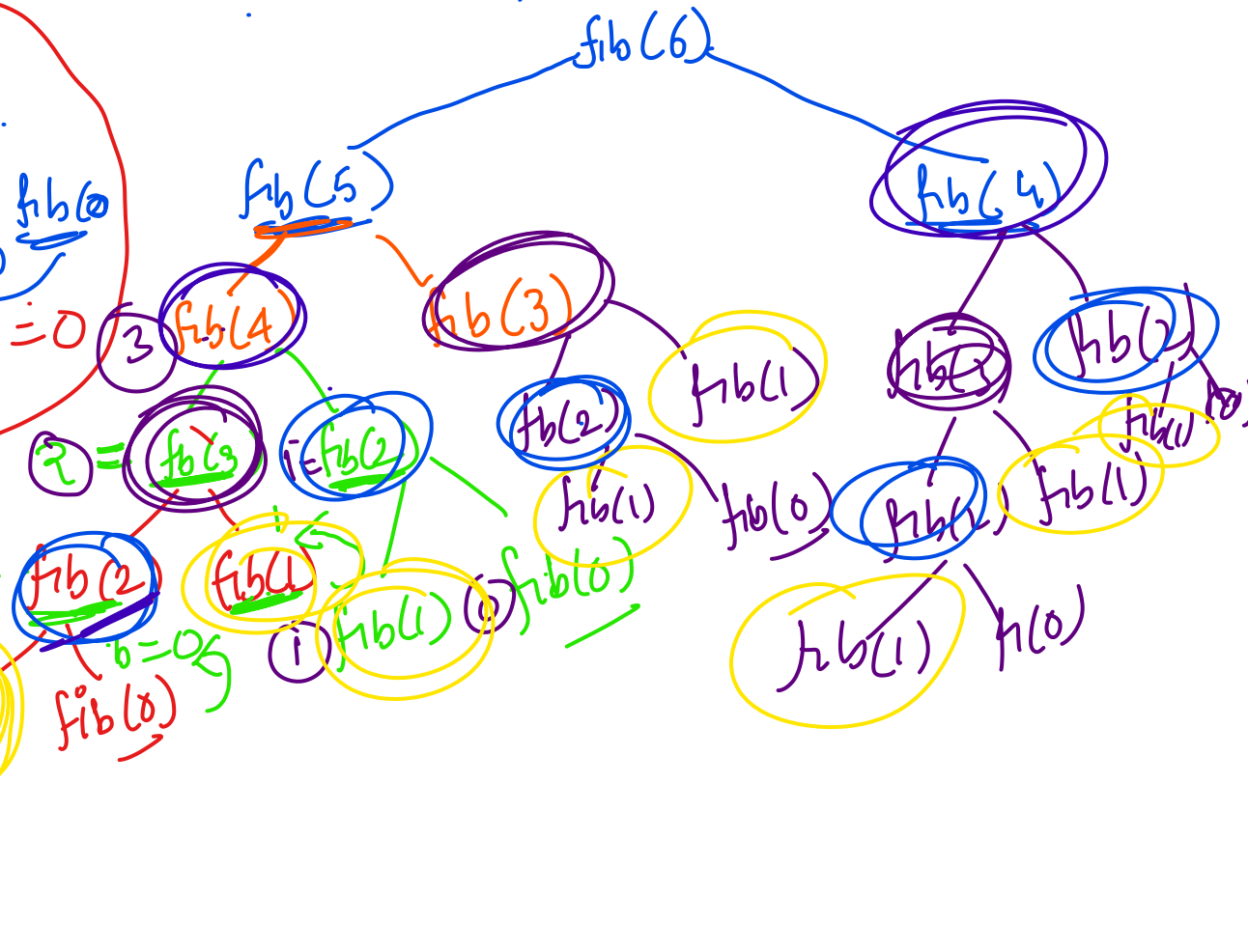
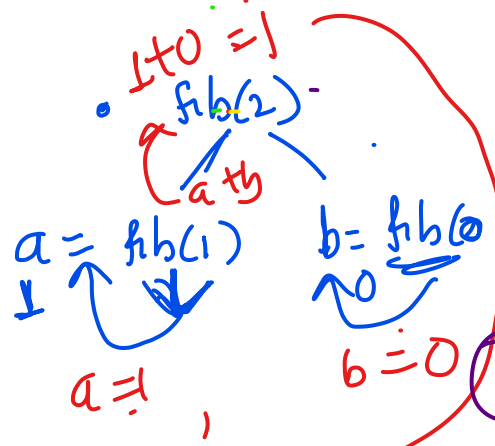
```
fun(a, b)
{
  if (a == 0) return 0;
  a = a + b;
  b = a - b;
  f(a, b);
  f(b, a);
}
```

0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25

$fib(0) = 0$, $fib(1) = 1$, $fib(2) = 1$

```

fib(n)
{
    if (n == 0 || n == 1)
        return n;
    a = fib(n-1);
    b = fib(n-2);
    return a + b;
}
    
```



$$T(n) = c + T(n-1) + T(n-2)$$

$$T(n) = T(n-1) + T(n-2) + 1$$

231

$$O(2^n)$$

Without DP :-

fib(n)

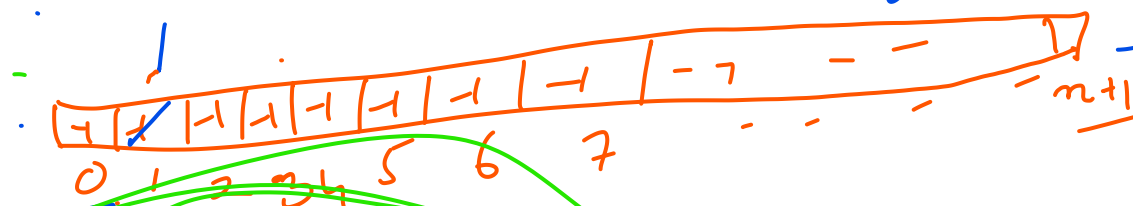
if (n == 0 || n == 1)
return n;

a = fib(n-1)

b = fib(n-2)

return a + b

With DP :- an array or table
we will make that will store fib(4)
(calculated sum) $T[n+1] = \{ -1, 3, \text{fib}(3), \text{fib}(2) \}$



fib(n)

if (n == 0 || n == 1)

{ T[n] = n

return T[n]

if (T[n-1] == -1)

T[n-1] = fib(n-1)

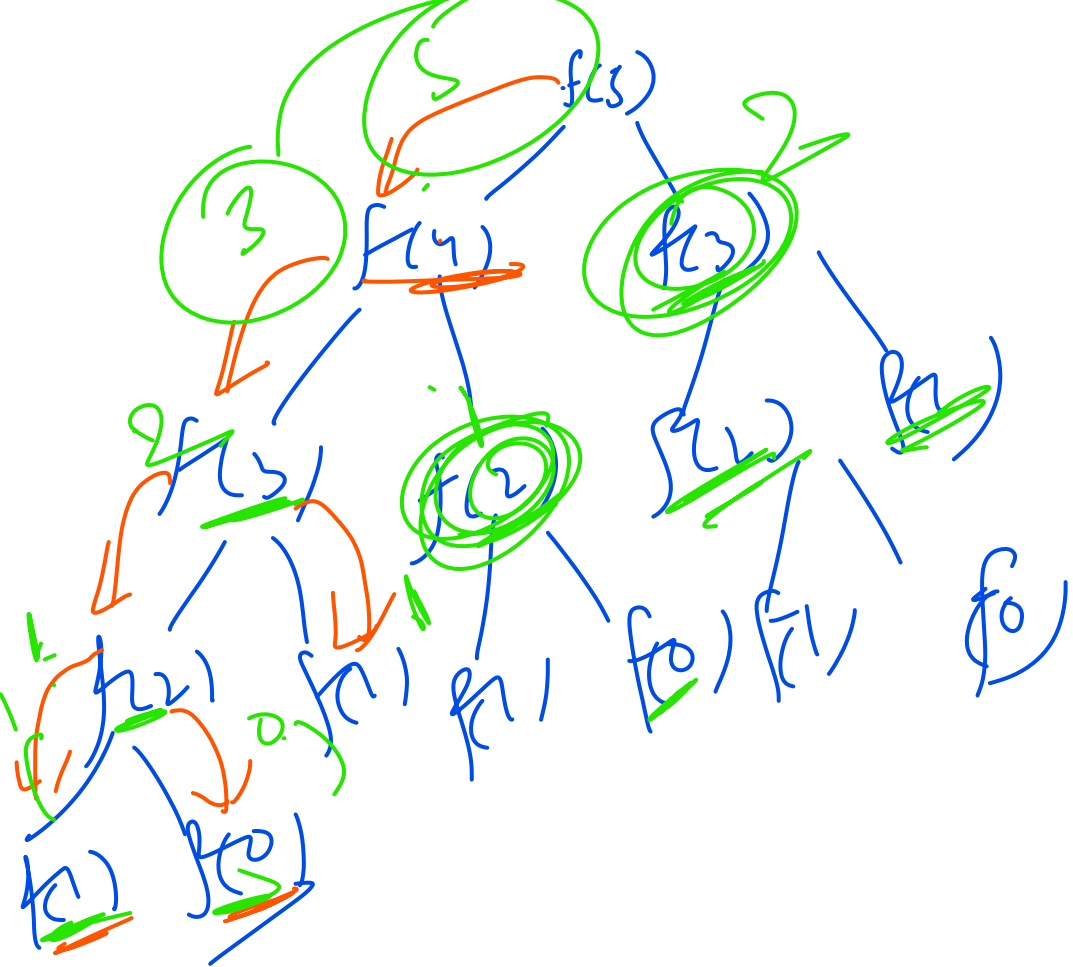
if (T[n-2] == -1)

T[n-2] = fib(n-2)

T[n] = T[n-1] + T[n-2]

return T[n]

With DP



0	1	2	3	4	5
1	1	1	1	1	1
0	1	2	3	4	5

$$T(5) = T(4) +$$

$$T(4) = T(3) + T(0)$$

$$= 1$$

Thanks for water

